IIT Guwahati - Department of Computer Science & Engineering

CS 2201 – Computer Architecture & Organization

Quiz #1 - (21.08.2026: 90 minutes, 40 marks)

Index the facing sheet with question numbers and corresponding page numbers.

Q. No	Q:1	Q:2	Q:3	Q:4	Q:5	Q:6	Q:7	Q:8	Q:9	Q:10	Total
Max. Marks	3	3	4	4	4	4	4	4	5	5	40

1. Answer the following.
   (a) Name India's indigenous microprocessor developed by IIT Madras.
   (b) Name the RISC processor brand by Qualcomm used in mobile phones.
   (c) What is the full form of MIPS? (MIPS is a RISC processor)
   (d) Who is the union cabinet minister of Government of India for the ministry of education?
   (e) Which Indian state houses Ratan Tata Electronics City?
   (f) Which date is celebrated as the Foundation Day of IIT Guwahati?

Ans: **(0.5 marks each)**

a. Shakti
b. Snapdragon
c. Microprocessor without interlocked pipelined stages
d. Shri Pralhad Joshi
e. Assam
f. 1 September

2. Match the following with unique mapping from Set A to Set B.

Set A	Set B
(a) RAW Hazard	(p) Operand forwarding
(b) WAR Hazard	(q) Resource duplication
(c) WAW Hazard	(r) Anti dependence
(d) Control Hazard	(s) Delay slot
(e) Structural Hazard	(t) Software scheduling
(f) Load-ALU dependency	(u) Output dependence

Ans: **a-p,**
b-r,
c-u,
d-s,
e-q,
f-t **(0.5 marks each)**

3. Illustrate the division of 14 by 5 using restoring and non-restoring division.

Ans: Restoring Division Algorithm: (0.4 marks for each step/iteration)

Input Parameters		**Register Configuration**		**Initial Register Values**	
Dividend (14)	01110_2 (5-bit)	**A**	Remainder Register (5-bit)	**A**	00000
Divisor (5)	00101_2 (5-bit)	**Q**	Quotient Register (5-bit)	**Q**	01110
n (bits in Q)	5	**M**	Divisor Register (5-bit)	**M**	00101 (Iterations n = 5)

Step (count)	**Actions**	**n**	**A (Remainder)**	**Q (Quotient)**	**M (Divisor)**	**Visual : Shift Left (A, Q)**	**Remarks**
5	Initialize	5	00000	01110	00101	A: 00000 | Q: 01110	Initial values
4	Shift left (A, Q)	5	00000	11100	00101	A: 00000 | Q: 01110 → A: 00000 | Q: 11100	Shift left (MSB of Q → LSB of A)
	A = A - M	5	11011	11100	00101		11011 = -5 (Negative)
	Restore A, set $Q_0 = 0$	4	00000	11100	00101		Restore, Set $Q_0 = 0$
3	Shift left (A, Q)	4	00001	11000	00101	A: 00000 | Q: 11100 → A: 00001 | Q: 11000	Shift left (MSB of Q → LSB of A)
	A = A - M	4	11100	11000	00101		11100 = -4 (Negative)
	Restore A, set $Q_0 = 0$	3	00001	11000	00101		Restore, Set $Q_0 = 0$
2	Shift left (A, Q)	3	00011	10000	00101	A: 00001 | Q: 11000 → A: 00011 | Q: 10000	Shift left (MSB of Q → LSB of A)
	A = A - M	3	11110	10000	00101		11110 = -2 (Negative)
	Restore A, set $Q_0 = 0$	2	00011	10000	00101		Restore, Set $Q_0 = 0$
1	Shift left (A, Q)	2	00111	00000	00101	A: 00011 | Q: 10000 → A: 00111 | Q: 00000	Shift left (MSB of Q → LSB of A)

Step (count)	Actions	n	A (Remainder)	Q (Quotient)	M (Divisor)	Visual : Shift Left (A, Q)	Remarks
	A = A - M	2	00010	00000	00101		00010 = +2 (Positive)
	Set $Q_0 = 1$	1	00010	00001	00101		Set $Q_0 = 1$
0	Shift left (A, Q)	1	00100	00010	00101	A: 00010 | Q: 00001 → A: 00100 | Q: 00010	Shift left (MSB of Q → LSB of A)
	A = A - M	1	11111	00010	00101		11111 = -1 (Negative)
	Restore A, set $Q_0 = 0$	0	00100	00010	00101		Set $Q_0 = 0$ (Done)

Final Result:
Quotient (Q) = 000102 = 2_{10}

Remainder (A) = 001002 =4_{10}

b. Non-Restoring Division Algorithm: (0.4 marks for each step/iteration)

Input Parameters		**Register Configuration**		**Initial Register Values**	
Dividend (14)	01110_2 (5-bit)	**A**	Accumulator / Remainder Register (5-bit)	**A**	00000
Divisor (5)	00101_2 (5-bit)	**Q**	Quotient Register (5-bit)	**Q**	01110
n (bits in Q)	5	**M**	Divisor Register (5-bit)	**M**	00101
		-M	2's Complement of M = 11011_2	**-M**	11011 (Iterations n = 5)

Step (count)	Actions	n	A (Accumulator)	Q (Quotient)	M (Divisor)	Visual : Shift Left (A, Q)	Remarks
5	Initialize	5	00000	01110	00101	A: 00000 | Q: 01110	Initial values
4	1. Shift left (A, Q)	5	00000	11100	00101	A: 00000 | Q: 01110 → A: 00000 | Q: 11100	Shift left (MSB of Q → LSB of A)
	2. Since A ≥ 0, A = A - M	5	11011	11100	00101		11011 = -5 (Negative)

Step (count)	Actions	n	A (Accumulator)	Q (Quotient)	M (Divisor)	Visual : Shift Left (A, Q)	Remarks
	3. Since A < 0, Set $Q_0 = 0$	4	11011	11100	00101		Set $Q_0 = 0$
3	1. Shift left (A, Q)	4	10111	11000	00101	A: 11011 | Q: 11100 → A: 10111 | Q: 11000	Shift left (MSB of Q → LSB of A)
	2. Since A < 0, A = A + M	4	11100	11000	00101		11100 = -4 (Negative)
	3. Since A < 0, Set $Q_0 = 0$	3	11100	11000	00101		Set $Q_0 = 0$
2	1. Shift left (A, Q)	3	11001	10000	00101	A: 11100 | Q: 11000 → A: 11001 | Q: 10000	Shift left (MSB of Q → LSB of A)
	2. Since A < 0, A = A + M	3	11110	10000	00101		11110 = -2 (Negative)
	3. Since A < 0, Set $Q_0 = 0$	2	11110	10000	00101		Set $Q_0 = 0$
1	1. Shift left (A, Q)	2	11101	00000	00101	A: 11110 | Q: 10000 → A: 11101 | Q: 00000	Shift left (MSB of Q → LSB of A)
	2. Since A < 0, A = A + M	2	00010	00000	00101		00010 = +2 (Positive)
	3. Since A ≥ 0, Set $Q_0 = 1$	1	00010	00001	00101		Set $Q_0 = 1$
0	1. Shift left (A, Q)	1	00100	00010	00101	A: 00010 | Q: 00001 → A: 00100 | Q: 00010	Shift left (MSB of Q → LSB of A)
	2. Since A ≥ 0, A = A - M	1	11111	00010	00101		11111 = -1 (Negative)
	3. Since A < 0, Set $Q_0 = 0$	0	11111	00010	00101		Set $Q_0 = 0$
	Final Correction: Since A < 0, A = A + M	0	00100	00010	00101		Restore Remainder : 11111 + 00101 = 00100

Final Result:

(a) Quotient (Q) = $00010_2 = 2_{10}$

(b) Remainder (A) = $00100_2 = 4_{10}$

4. Illustrate the multiplication of -15 (multiplicand) and 10 (multiplier) using Booth's algorithm.

Ans: -15 x 10 = 10001 x 01010

Q = 01010 (multiplier)
M = 10001 (multiplicand)
-M = 01111

Iteration	Step	M / -M	A	Q	Q-1
0	Initial values	10001 / 01111	00000	01010	0
1	00 → No op	10001 / 01111	00000	01010	0
(0.8 mark)	shift right		00000	00101	0
2	10 → A = A - M	10001 / 01111	01111	00101	0
(0.8 mark)	shift right		00111	10010	1
3	01→ A = A + M	10001 / 01111	11000	10010	1
(0.8 mark)	shift right		11100	01001	0
4	10 → A = A - M	10001 / 01111	01011	01001	0
(0.8 mark)	shift right		00101	10100	1
5	01 → A = A + M	10001 / 01111	10110	10100	1
(0.8 mark)	shift right		11011	01010	0

Final Answer: **11011 01010 = $(-150)_{10}$**

Verification:

Since the MSB is 1, the number is negative in 2's complement representation.
Now, find its 2's complement to obtain the magnitude:

```
00100 10101
+           1
-----------
00100 10110
```

00100 10110 = $(150)_{10}$

Therefore, 11011 01010 = $(-150)_{10}$

Hence, the result is verified.

5. Let A, B and C are registers that holds the values -120, 97 and 85 in 8 bits 2-s complement form. Two operations (i) A x B and (ii) A x C is done using Booth's multiplication. Will the execution time of the generating the product in above two operations same? Why/Why not?

Ans: Booth Multiplication: Execution Time Comparison

Multiplicand A (−120) = 10001000

Multiplier B (97) = 01100001

Multiplier C (85) = 01010101

(i) A × B.

Multiplier Q with appended Q−1 = 011000010

Q, Q−1 bit transitions: 10, 01, 00, 00, 00, 10, 11, 01

Operations: 4 arithmetic operations (A ← A − M or A ← A + M) and 4 NO OPS.

Total: 4 partial products. **(1 mark)**

(ii) A × C

Multiplier Q with appended Q−1 = 010101010

Q, Q−1 bit transitions: 10, 01, 10, 01, 10, 01, 10, 01

Operations: 8 arithmetic operations (A ← A − M or A ← A + M) and 0 NO OPS.

Total: 8 partial products. **(1 mark)**

No. execution time will not be the same. Operation (ii) generates 8 partial products, whereas operation (i) generates 4 partial products. **(2 marks)**

6. (a) Express -625.625 in IEEE 754 form an write its content in hexadecimal format.

(b) If the IEEE 754 representation of a single precision float is 0x43FF8800, then what is the number?

Ans: (a)

625= 1001110001

0.625 = 0.101

625.625 = 1001110001.101

= $1.001110001101 * 2^9$ **(½ mark)**

Biased exp = 9 + 127

= 136

= 10001000 **(½ mark)**

Sign bit = 1

1	10001000	00111000110100000000000

(½ mark)

In hexadecimal format,

0xC41C6800 **(½ mark)**

(b)

In decimal form,

0100 0011 1111 1111 1000 1000 0000 0000 **(½ mark)**

1	10000111	11111111000100000000000

E' = 10000111

= 135

E = 135 - 127

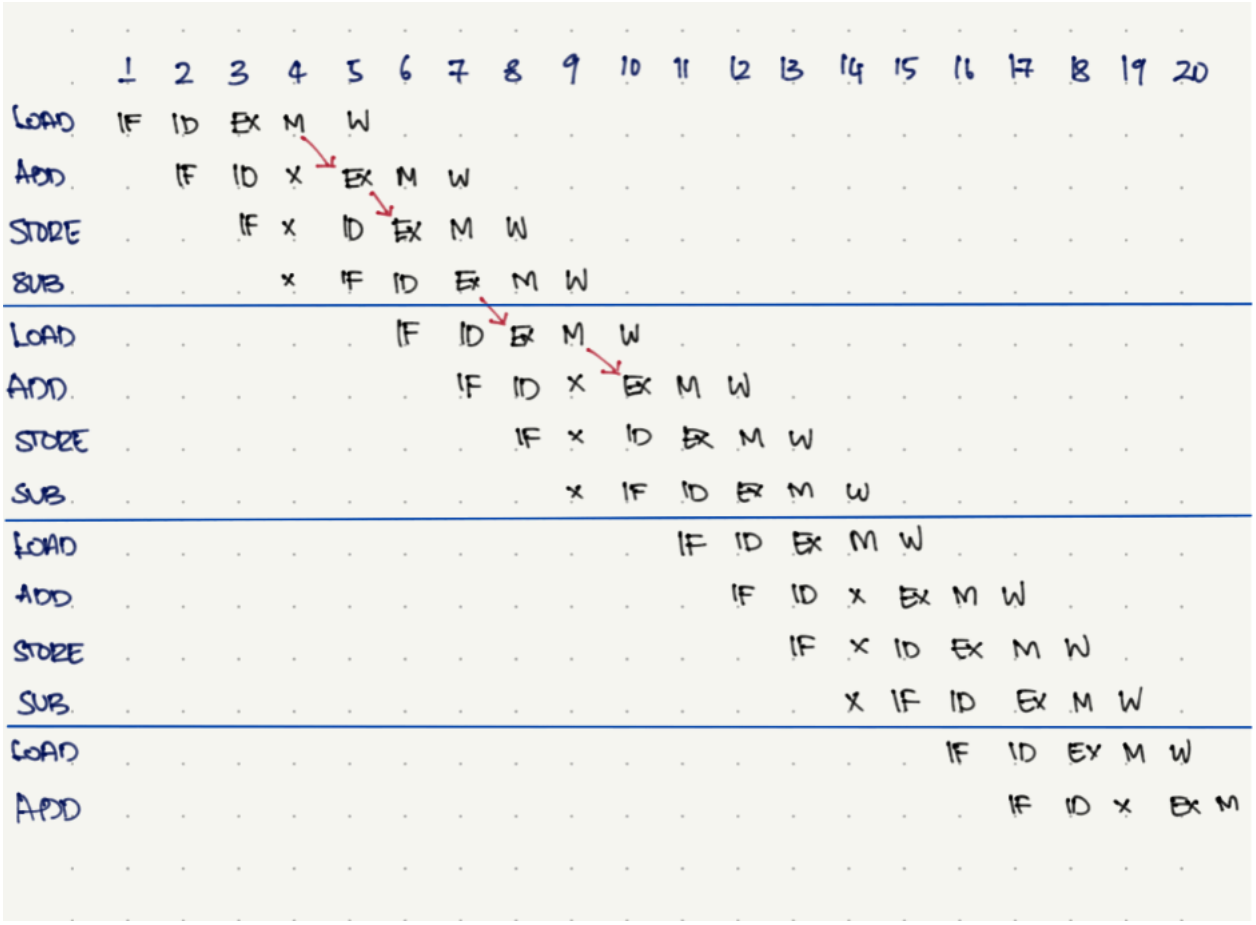
= 8 **(½ mark)**

Value = + 1.111111110001 * 2^8 **(½ mark)**

= 511 + 0.625

= + 511.0625 **(½ mark)**

7. Consider a 5-stage in-order RISC instruction pipeline (IF-ID-EX-MEM-WB). Consider a program that has a sequence of instructions of the form LOAD, ADD, STORE, SUB that is repeated 100 times in that order. So, the program has a total of 400 instructions. The ADD has a data dependency on the value accessed from memory by the immediate LOAD before it. The STORE and SUB have a data dependency on the resultant of the ADD instruction immediately preceding them. Every LOAD instruction (except the first instruction) has a data dependency on the resultant of immediate SUB before it. Operand forwarding is done to handle data hazards. Assume that the IF stage for the 1st LOAD is done at clock cycle number 101.
   (a) What is the 298th instruction?
   (b) In which clock cycle number, the 60th STORE performs MEM stage?
   (c) What is the average CPI (cycles/instruction) for this program?



[pipelining 1 mark]

(a) In. 298 = 298%4 => 2 => ADD Instruction **[1 mark]**

(b) STORE performs MEM in this order => 107, 112, 117, ... **[1 mark]**

60th STORE => 107 + (60-1)x5 = 402

(c) Average CPI = (109 + (99*5) - 100) / 400 => 504/400 = 1.26 **[1 mark]**

8. Consider a scenario where registers R1 and R2 hold values 2000 and 3000, respectively. Memory location whose addresses are 2000, 3000, 5000 and 6000 hold values 5000, 6000, 7000 and 8000, respectively. What will be the value of R3 after executing the ADD instruction in the following 4 independent cases?

   (a) ADD R3, R2, #100 // Immediate addressing mode

   (b) ADD R3, R1, 3000 // Direct addressing mode

   (c) ADD R3, [R1], R2 // Register indirect addressing mode

   (d) ADD R3, R2, [3000] // Memory indirect addressing mode

   R1 = 2000

   R2 = 3000

Memory Address	Content
2000	5000

3000	6000
5000	7000
6000	8000

a. R3 = 3000 + 100 = 3100 **[1 mark]**

b. R3 = 2000 + M[3000] = 2000 + 6000 = 8000 **[1 mark]**

c. R3 = [R1] + 3000 = M[3000] = 3000 = 8000 **[1 mark]**

d. R3 = 3000 + M[M[3000]] = 3000 + M[6000] = 3000 + 8000 = 11000 **[1 mark]**

9. Given a non-pipelined instruction execution design (P) running at 1.8 GHz, that takes 5 cycles to finish an instruction. The equivalent 5-stage pipelined design (Q) operates only at 1.5 GHz due to additional latching overhead. Due to structural hazard, 10% of floating-point instructions cause a stall of 5 cycles each. All unconditional branches and all LOAD-ALU instruction combinations cause a stall of 1 cycle each. For conditional branches one fourth of them do not create any stall whereas the rest of them create a stall of 1 cycle each. In a given program, on an average for every unconditional branch there are four conditional branch instructions. Assume that the program has 20% floating-point instructions, 30% branch instructions and 25% of instructions are LOAD-ALU combinations. Assume that no other instructions or its combinations incur stalls. What is the speedup of pipelined design over the non-pipelined design if there are no stalls for a non-pipelined architecture?

Ans: unpipelined

CPI=5, CCT = 1 / 1.8 x 10^9

pipelined

CCT= 1 / 1.5 x 10^9

For FP instructions,

FP stall = 5 * 10/100 * 20/100 = 0.10

For LOAD ALU instructions,

LOAD-ALU stall = 1 * 25/100 = 0.25

For Branch,

let unconditional be x, then conditional = 4x

=> 5x = 30%

=> x= 6%

Therefore, unconditional = 6%, conditional =24%

Now,

Conditional stall = 24/100 * ¾ * 1 = 0.18

Unconditional stall = 6/100 * 1 = 0.06

=> Branch stall = 0.18 + 0.06 = 0.24 **[3 marks, stall CPI]**

Now, pipelined CPI = Base CPI + stall CPI

= 1 + FP stall + LOAD-ALU stall + Branch stall

= 1 + 0.10 + 0.25 + 0.24 = 1.59 **[1 mark]**

SpeedUp = Execution Time unpipelined / Execution Time pipelined

= CPIup * CCTup / CPIp * CCTp

= $(5 * 1/1.8 * 10^9) / (1.59 * 1/1.5 * 10^9)$

= 2.62 **[1 mark]**

10. A processor with 128 registers uses 4B instruction format. It has an addressability of 4GB memory, and it operates on segmented memory where each memory segments is 64KB. The processor has broadly three types of instructions; jump, register, immediate. Jump instruction has 2 register operands and an offset field. Each jump instruction is operated by looking at value of two register operands. There are three jump opcodes (less than, greater than, equal to). The offset field indicates the relative distance (in bytes) of the target address with respect to current program counter value. Target address of a jump is always within the current memory segment. There are 256 register instructions each with 3 register operands. Each immediate instruction takes a register operand and a 16-bit scalar value.
    (a) What is the maximum number of unique opcodes possible for immediate instruction?
    (b) Design an instruction encoding format for the above instruction set architecture clearly indicating the opcode and operands fields for all three types of instructions.
    (c) Rather than 3 unique jump opcodes, if there are only 2 unique jump opcodes, and all other instructions and its format remain unchanged, what is the maximum number of unique opcodes possible for immediate instruction? – Show the revised encoding pattern like (b) above.

Ans: 128 registers = 2^7

4B = 32-bit instruction format

Memory segment = 64 KB = 2^{16} B = Offset

Jump:

Opcode (2 bits)	R (7 bits)	R (7 bits)	Offset (16 bits)

Register:

Opcode (11 bits)	R (7 bits)	R (7 bits)	R (7 bits)

Immediate:

Opcode (9 bits)	R (7 bits)	Scalar (16 bits)

For J,

Number of Opcodes= 3

XX

00

01

10

11 → Free

For R,

Number of Instr. = 256, need 8 bits

11 0 XXXX XXXX

Remaining bits → 111

For I,

X XXXX XXXX

111 XX XXXX

(a) => Number of possible instrs = $2^6 = 64$ **[1 mark]**

(b) Given above **[2 marks]**

(c)

Now, if only 2 opcodes for J type,

Then,

For J, XX

00

01

10

11 → Free

For R,

11 0 XXXX XXXX

For I,

X XXXX XXXX

Since, 10 & 11 (110 is taken) are free.

111 XX XXXX => $2^6 = 64$

10 XXX XXXX => $2^7 = 128$

=> Total possible opcodes = 64 + 128 = 192 **[2 marks]**